

تقرير الهندسة البرمجية وتكامل الأنظمة

Software Engineering Integration Report

حالة المشروع الحالية:

الموقع منتهي مبدئياً من ناحية الواجهات (Frontend) والبنية الأساسية بعد نجاح البناء محلياً وربط إعدادات البيئة و Supabase. المشروع ليس منتجاً نهائياً (Production-ready) حتى يتم بناء واختبار تكاملات الفنادق، الطيران، والرحلات الداخلية، وربطها بالكامل مع CRM وملف العميل، وآلية الاعتماد اليدوي للدفع End-to-End.

1. نطاق المشروع (Project Scope)

المشروع عبارة عن منصة سفر وحجوزات متكاملة تتكون من الأنظمة والخدمات التالية:

- موقع عام: لعرض الخدمات، البحث، وإدارة تجربة المستخدم.
- حسابات العملاء: بوابة مخصصة لمتابعة الحجوزات والطلبات والمستندات.
- نظام CRM داخلي: للإدارة والتنفيذ المباشر والمتابعة المالية والتشغيلية.
- قاعدة بيانات: إدارة المستخدمين والحجوزات مبنية على Node.js و Supabase.
- تكاملات خارجية: جلب بيانات الفنادق والطيران المباشرة عبر APIs.
- رحلات وبرامج سياحية: رحلات يتم إدخالها عبر نظام الإدارة أو سحبها من نظام العميل الخارجي.
- دورة حجز متكاملة: تبدأ بطلب العميل → دفع خارج النظام → مراجعة واعتماد الإدارة → التأكيد النهائي.

2. المكونات الرئيسية للنظام (System Components)

2.1 الموقع العام (Public Web)

- الصفحة الرئيسية واستعراض الخدمات.
- محركات البحث للرحلات، الفنادق، والبرامج السياحية.
- صفحة خدمات التأشيرات والرحلات القصيرة.
- تسجيل الدخول، إنشاء الحساب، وصفحة تفاصيل الحجز وتتبع حالته.

2.2 بوابة العميل (Client Portal)

- إدارة الملف الشخصي والسجل التاريخي والحجوزات الحالية.
- إرسال طلبات الحجز ورفع المستندات والوثائق المطلوبة.
- استلام الإشعارات والتنبيهات المباشرة من الإدارة.
- تتبع حالة الطلبات: (في انتظار الدفع، المراجعة، التأكيد، أو الإلغاء).

2.3 لوحة الإدارة ونظام الـ CRM (Admin & CRM)

- لوحة تحكم شاملة لإدارة العملاء وطلبات الحجز والخدمات.
- مراجعة المستندات وتأكيد الدفع اليدوي بعد استلام التحويلات.
- اعتماد الحجوزات أو رفضها وإبداء الأسباب.
- إدارة العروض والبرامج السياحية وتحديث أسعارها وتوافرها.
- سجلات النظام (Logs) ومراقبة الأخطاء وإرسال الإشعارات.

3. نموذج البيانات الأساسي (Data Model Architecture)

يعتمد النظام على مصدر واحد للحقيقة (Single Source of Truth) لتفادي تضارب البيانات عبر مختلف الشاشات والخدمات.

3.1 بيانات المستخدمين (Users)

- **auth.users**: الجداول الخاصة بالمصادقة وإدارة الهوية.
- **profiles**: البيانات الشخصية للعميل أو الموظف (**profiles.id** للملف الداخلي، و **profiles.user_id** للربط بنظام Auth).
- **organizations**: بيانات المؤسسات أو الشركات (عند التوسع لدعم متعدد المؤسسات (Multi-tenancy)).

3.2 الطلبات والحجوزات (Travel Requests)

اعتماد جدول كيان موحد للحجز باسم **travel_requests** ترتبط به جداول التفاصيل المخصصة:

- **hotel_booking_details**
- **flight_booking_details**
- **trip_booking_details**
- **visa_booking_details**

🔗 فائدة التصميم الموحد:

يضمن هذا التصميم وجود شاشة موحدة وحالات مشتركة وتقارير مالية وتنفيذية متكاملة داخل CRM وللمعمل، بدلاً من فصل الحجوزات إلى أنظمة مستقلة.

3.3 الفصل بين مصادر البيانات (Data Provenance)

التمييز الحازم بين البيانات الخارجية المجلوبة عبر API والبيانات المدخلة يدوياً بواسطة الأدمن عبر تخصيص الحقول التالية:

```
source VARCHAR(50),           -- 'external_api' | 'manual_admin'
external_provider VARCHAR(50), -- 'amadeus' | 'hotel_x' | 'client_system'
external_id VARCHAR(100),      -- Unique ID from provider
last_synced_at TIMESTAMP,      -- Sync timestamp
raw_payload JSONB              -- Full original response snapshot
```

4. الخدمة الأولى: تكامل الفنادق مع CRM (Hotels Integration)

4.1 وصف الخدمة

ربط المنصة بنظام الفنادق الخارجي الموفر من العميل لتنفيذ الاستعلام المباشر عن الأسعار والغرف والمواصفات وإنشاء طلب الحجز تلقائياً داخل الـ CRM.

4.2 قائمة البيانات المطلوبة من نظام العميل

- [] رابط API الأساسي (Base URL) لبيئتي الاختبار والإنتاج (Sandbox & Production).
- [] آلية المصادقة (API Key / Bearer Token / OAuth2).
- [] التوثيق الفني لكافة الـ Endpoints (البحث، التوافر، الحجز، الإلغاء).
- [] هيكل البيانات الخاص بأكواد الفنادق، الغرف، العروض، وشروط الإلغاء.
- [] توضيح ضوابط الأسعار (شاملة الضرائب أم لا، العملة، المنطقة الزمنية).
- [] حدود معدل الاستخدام (Rate Limits) وسياسة التعامل مع الأخطاء.

4.3 طبقة المحول (HotelProviderAdapter)

فصل تفاصيل الـ API الخارجي عن واجهة المستخدم عبر بناء طبقة محول موحدة:

```
type HotelSearchResult = {
  provider: string;
  externalHotelId: string;
  hotelName: string;
  city: string;
  country: string;
  rating: number | null;
  rooms: HotelRoomResult[];
  currency: string;
  sourceUpdatedAt: string | null;
}

type HotelRoomResult = {
  externalRoomId: string;
  roomType: string;
  pricePerNight: number;
  totalPrice: number;
  isAvailable: boolean;
  cancellationPolicy: string;
}
```

4.4 دورة حجز الفندق (Hotel Booking Lifecycle)

بحث العميل ← عرض النتائج ← إعادة التحقق (Re-check) ← إنشاء طلب (pending_payment) ← تحويل مالي خارجي ← مراجعة وتأكيد الأدمن ← تأكيد الحجز الخارجي (confirmed) ← إشعارات العميل.

4.5 حالات الفشل والتغلب عليها

- تغيير السعر أو نفاذ الغرف: تنفيذ عملية Re-check إجبارية قبل حفظ الطلب النهائي.
- تكرار إرسال الحجز: استخدام idempotency_key لمنع الحجز المزدوج.
- فشل API الخارجي بعد الدفع: وضع الطلب في حالة manual_action_required للتنفيذ اليدوي بواسطة الأدمن.

5. الخدمة الثانية: تكامل الطيران (Flights Integration)

5.1 وصف الخدمة

جلب الرحلات الجوية العالمية، معالجة الأسعار والمواعيد، وعرضها للعملاء مع إتاحة حجزها وربطها بالـ CRM.

5.2 الأدوات التقنية للبنية التحتية

الأداة / التقنية	الاستخدام والهدف
Official Flight APIs	الخيار الأساسي والأمن المعتمد للحصول على البيانات والحجوزات (Amadeus, Duffel, etc).
Redis Cache	تخزين مؤقت لنتائج البحث الشائعة لتخفيف الضغط وتقليل تكاليف الاستعلام.

⚠ تنبيه قانوني وتشغيلي بخصوص Scraping:

الاعتماد على Scraping في بيانات الإنتاج (Production) ينطوي على مخاطر عالية (تغير الواجهات، الحظر، الأبعاد القانونية).
التوصية الفنية المعتمدة هي استخدام APIs رسمية مرخصة. كما يحظر تشغيل Playwright/Puppeteer داخل Vercel Serverless Functions نظراً لقيود الذاكرة وزمن التنفيذ.

5.3 نموذج رحلة الجو الموحد (Normalized Flight Result)

```
type FlightSearchResult = {
  provider: string;
  externalOfferId: string;
  carrier: string;
  flightNumber: string | null;
  origin: string;
  destination: string;
  departureAt: string;
  arrivalAt: string;
  durationMinutes: number | null;
  stops: number;
  cabinClass: string;
  price: number;
  currency: string;
  baggage: BaggageRule | null;
  expiresAt: string | null;
}
```

6. الخدمة الثالثة: الرحلات والبرامج القصيرة (Short Trips & Packages)

6.1 وصف الخدمة

إدارة وتأكيذ البرامج السياحية والرحلات الداخلية التي لا تعتمد بالضرورة على APIs عالمية، بل يتم إدخالها يدوياً من الأدمن أو سحبها من نظام العميل.

```

type TripPackage = {
  id: string;
  externalId: string | null;
  name: string;
  destination: string;
  startDate: string;
  endDate: string;
  durationNights: number;
  description: string | null;
  hotelName: string | null;
  inclusions: string[];
  exclusions: string[];
  itinerary: Record<string, any>;
  pricePerPerson: number;
  currency: string;
  availableSpots: number;
  status: "draft" | "published" | "sold_out" | "expired";
  source: "admin" | "external_api";
}

```

🔒 التحكم بالتزامن (Concurrency Control):

خصم المقاعد المتاحة (availableSpots) يجب أن يتم بصورة ذرية (Atomic Operation) في قاعدة البيانات لمنع Overbooking عند الحجز المزدوج في نفس اللحظة.

7. دورة المعاملات المالية والدفع اليدوي (Payment Architecture)

7.1 سير عمل الدفع اليدوي الحالي

إنشاء الحجز ← حالة pending_payment ← تحويل مالي خارجي من العميل ← رفع الإيصال ← مراجعة الإدارة ← تأكيد الدفع ← تأكيد الحجز النهائية.

7.2 هيكل سجل المعاملات المالية (payment_records)

```

CREATE TABLE payment_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  travel_request_id UUID REFERENCES travel_requests(id),
  client_id UUID REFERENCES profiles(id),
  amount DECIMAL(10, 2) NOT NULL,
  currency VARCHAR(3) DEFAULT 'EGP',
  payment_method VARCHAR(50),          -- 'bank_transfer' | 'vodafone_cash' | 'card'
  transaction_reference VARCHAR(100),
  receipt_attachment_url TEXT,
  status VARCHAR(30) DEFAULT 'submitted', --
  'submitted' | 'under_review' | 'verified' | 'rejected'
  verified_by UUID REFERENCES profiles(id),

```

```
verified_at TIMESTAMP,  
rejection_reason TEXT,  
created_at TIMESTAMP DEFAULT NOW()  
);
```

8. نظام الحالات الموحد (System Status Engine)

فصل حالة سير المستندات عن حالة دورة الحجز والمالية منعاً للتعارض والتداخل في لوحة التحكم:

- حالة سير المستندات: pending_documents → documents_review → docs_approved → completed
- حالة دورة الحجز والمالية: draft → pending_payment → payment_under_review → confirmed → completed

9. هيكلية طبقات النظام (Architecture Layers)

```
src/  
├── app/                # Presentation Layer (Next.js App Router Pages)  
├── app/api/            # API & Route Handlers Layer (Authentication & Validation)  
├── lib/  
│   ├── integrations/  # Integration Layer  
│   │   ├── hotels/    # Hotel Adapter, Client, Mapper & Services  
│   │   ├── flights/   # Flight Adapter, Client, Mapper & Services  
│   │   └── trips/     # Trip Sync & Package Services  
│   ├── db/            # Data Layer (Supabase / Database Repositories)  
│   └── jobs/          # Workers & Background Jobs Layer (Cron & Queues)
```

10. المتغيرات البيئية السرية (Environment Variables)

```
# Database & Auth  
SUPABASE_URL=https://your-project.supabase.co  
SUPABASE_SERVICE_ROLE_KEY=your-super-secret-service-role-key  
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co  
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key  
  
# Hotel Provider Integration  
HOTEL_PROVIDER_BASE_URL=https://api.hotelprovider.com/v1  
HOTEL_PROVIDER_API_KEY=your_hotel_api_key  
HOTEL_PROVIDER_SECRET=your_hotel_secret  
  
# Flight Provider Integration  
FLIGHT_PROVIDER_BASE_URL=https://api.flightprovider.com/v1  
FLIGHT_PROVIDER_API_KEY=your_flight_api_key  
  
# Infrastructure & Queues  
REDIS_URL=redis://default:password@localhost:6379  
WEBHOOK_SIGNING_SECRET=whsec_your_webhook_secret
```

11. متطلبات الأمان والتأمين (Security Matrix)

- حماية المفاتيح: منع استدعاء APIs الخارجية من المتصفح مباشرة، وحصرها في Server Components / API Routes.
- صلاحيات الوصول (Authorization): تفعيل Row Level Security (RLS) للتحقق من أن العميل يقرأ بياناته فقط، والأدمن يملك صلاحية الاعتماد المالي.
- التحقق من الأسعار: إعادة حساب الإجمالي في الخادم ومنع استلام الأسعار من مدخلات الواجهة (Frontend Input Injection).
- حماية البيانات الحساسة: تشفير المستندات وجوازات السفر المخزنة وعدم تسجيل بيانات حساسة داخل الـ Logs.

12. قائمة متطلبات العميل قبل بدء التكامل (Client Requirement Checklist)

- [] 1. توثيق API الفنادق والـ Base URL ورابط بيئة الاختبار (Sandbox).
- [] 2. بيانات الدخول ومفاتيح الاختبار (API Credentials).
- [] 3. قائمة أسلوب المصادقة وهيكل الطلبات والأخطاء (Error Handlers).
- [] 4. توضيح الآلية المتبعة لتأكيد الحجز النهائي أو إلغائه عبر API.
- [] 5. تحديد مزود بيانات الطيران المعتمد والترخيص القانوني.
- [] 6. توضيح مصدر الرحلات القصيرة (هل هي مدخلة يدوياً أم تسحب عبر API/Webhook؟).
- [] 7. تحديد سياسات الضرائب، العملات المعتمدة، والعمولات.
- [] 8. تحديد قائمة أدوار المستخدمين والموظفين داخل CRM وتوزيع الصلاحيات.

13. مراحل التنفيذ والتطوير (Implementation Roadmap)

- المرحلة 1: تثبيت الأساسيات: تثبيت قاعدة البيانات، نموذج travel_requests، والدفع اليدوي وصلاحيات CRM.
- المرحلة 2: تكامل الفنادق: بناء Hotel Adapter، البحث المباشر، Re-check، وربط الاعتماد اليدوي بالحجز الخارجي.
- المرحلة 3: الرحلات القصيرة: بناء إدارة البرامج السياحية (CRUD/Sync)، حساب الأسعار، وخصم المقاعد الذري.
- المرحلة 4: تكامل الطيران: ربط API الطيران الرسمي، إعداد Redis Cache، وإصدار PNR ومرجع الحجز.
- المرحلة 5: التكامل المالي الإلكتروني: ربط بوابة الدفع، Webhook Validation، والتأكيد الآلي للحجوزات.